



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Generics in Java



Generics - Introduction

- It is a mechanism by which a **single piece of code can manipulate different data types** without explicitly having a separate entity for each data type
- Generics were introduced in JDK 5.0
- **Generics** in Java is similar to **templates in C++**. The idea is to allow type (Integer, String, ... etc and user defined types) to be a parameter to methods, classes and interfaces.
- Generics aim of **reducing bugs** and adding an extra layer of **abstraction over types**.
- Generics add stability to your code by making more of your **bugs detectable at compile time**.

Why use Generics ?

1. Stronger type checks at compile time

- A Java compiler applies strong type checking to generic code and issues errors if the code **violates type safety**. Fixing **compile-time errors is easier than fixing runtime errors**, which can be difficult to find.

Why use Generics ?

2. Elimination of casts

- **Before Generics :**

```
1. List myIntegerList = new LinkedList();  
2. myIntegerList.add(new Integer(0));  
3. Integer x = (Integer) myIntegerList.iterator().next();
```

Note: Line no 3 if not properly typecasted will throw runtime exception

- **After Generics :**

```
1. List<Integer> myIntegerList = new LinkedList<Integer>();  
2. myIntegerList.add(new Integer(0));  
3. Integer x = myIntegerList.iterator().next();
```

Note: No need for typecasting here

Why use Generics ?

3. Enabling programmers to implement generic algorithms

- By using generics, programmers can implement generic algorithms that work on **collections of different types**, can be customized, and are **type safe and easier to read**.

Generics in Java

Generic Class

- A generic type is a **class or interface** that is parameterized over types. We use angle brackets (<>) to specify the type parameter.
- **Example** (non generics class)

//Java program to demonstrate the use of generics

```
class Demo{  
    private Object object;  
    public void set(Object object) {  
        this.object = object;  
    }  
    public Object get() {  
        return object;  
    }  
}
```

Note:

Since its methods accept or return an Object, we are free to pass in whatever we want, provided that it is not one of the primitive types. There is no way to verify, at compile time, how the class is used.

One part of the code may place an Integer in the set and expect to get Integers out of it, while another part of the code may mistakenly pass in a String Resulting in a runtime error.

Generics in Java

Generic Class

- A **generic class** is defined with the **following format**:

```
class name<T1, T2, ..., Tn> { /* ... */ }
```

- Delimited by angle brackets (<>), follows the class name.
- It specifies the **type parameters** (also called type variables) T1, T2, ..., and Tn.
- Generic class can have **multiple type parameters**.
- Type argument can also be a **custom type**.

Note:

- **In Parameter type we can not use primitives like 'int', 'char' or 'double'.**

Generics in Java

Generic Class – Example #1

- To update the **Demo class to use generics**,
 - Create a generic type declaration by changing the code “class Demo” to “class Demo<T>”.
 - This introduces the type variable, T, that can be used anywhere inside the class.

```
class Demo<T> {  
    // T stands for "Type"  
    private T t;  
    public void set(T t) {  
        this.t = t;  
    }  
    public T get() {  
        return t;  
    }  
}
```

Note:

All occurrences of Object are replaced by T.
A type variable can be any non-primitive type specified:

any class type,
any interface type,
any array type,
even another type variable.

This same technique can be applied to create generic interfaces.

Generic Class – Example #1

```
// Driver class
class GenericDemo {
    public static void main (String[] args) {
        // instance of Integer type
        Demo <Integer> iObj = new Demo<Integer>();
        iObj.set(25);
        System.out.println(iObj.get());

        // instance of String type
        Demo <String> sObj = new Demo<String>();
        sObj.set("Demo");
        System.out.println(sObj.get());
    }
}
```

Output:**25****Demo**

Type Parameter Naming Conventions

- The most **commonly used type parameter** names are:
 - E - Element (used extensively by the Java Collections Framework)
 - K - Key
 - N - Number
 - T - Type
 - V - Value
 - S,U,V etc. - 2nd, 3rd, 4th types

Generic Class – Example #2

// This example demonstrates user defined Generic classes

// We use < > to specify Parameter type

```
class Test<T> {  
    // An object of type T is declared  
    T obj;  
    Test(T obj) {  
        this.obj = obj;  
    }           // constructor  
    public T getObject() {  
        return this.obj;  
    }  
}
```

Generic Class – Example #2

// Driver class

```
class GenericDemo {  
    public static void main (String[] args) {  
        // instance of Integer type  
        Test <Integer> iObj = new Test<Integer>(15);  
        System.out.println(iObj.getObject());  
  
        // instance of String type  
        Test <String> sObj = new Test<String>("Test");  
        System.out.println(sObj.getObject());  
    }  
}
```

Output:

15

Test

Generic Class – Example #3

// This example demonstrates multiple type parameters in Java Generics

// We use < > to specify Parameter type

```
class Test<T, U, R> {  
  
    T ob1; // An object of type T  
  
    U ob2; // An object of type U  
  
    R ob3;  
  
    Test(T ob1, U ob2, R ob3)    {                // constructor  
  
        this.ob1 = ob1;  
  
        this.ob2 = ob2;  
  
        this.ob3=ob3;  
  
    }
```


Generics in Java

Generic Class – Example #3

```
public void print() {                // To print objects of T and U
    System.out.println(ob1);
    System.out.println(ob2);
    System.out.println(ob3);
}

}

// Driver class
class GenericDemo {
    public static void main (String[] args) {
        Test <String, Integer, Float> obj = new Test<>("Test", 15, 3.14f);
        obj.print();
    }
}
```

Output:

Test

15

3.14

Generics in Java

Java Generic Method

- Sometimes we don't want the whole class to be parameterized, in that case, we can create **Java generics method**.
- Generic methods are methods that introduce their **own type parameters**.
- This is similar to declaring a generic type, but the type parameter's **scope is limited to the method** where it is declared.
- **Static** and **non-static generic methods** are allowed.
- Since the constructor is a special kind of method, we can use **generics type in constructors too**.

Generic Method – Example #1

// This example demonstrates user defined Generic methods

```
class GenericMethod {  
    static <T> void genericDisplay (T element) {    // A Generic method example  
        System.out.println(element);  
        System.out.println(element.getClass().getName() +" = " + element);  
    }  
    public static void main(String[] args) {        // Driver method  
        // Calling generic method with Integer argument  
        genericDisplay(11);  
        // Calling generic method with String argument  
        genericDisplay("Test");  
        // Calling generic method with double argument  
        genericDisplay(1.0);  
    }  
}
```

Output:

11

java.lang.Integer = 11

Test

java.lang.String = Test

1.0

java.lang.Double = 1.0

Generic Class – Example #2

// This example demonstrates user defined Generic methods

```
public class GenericMethod {  
    public static < X > void printArray( X[] inputArray ) {      // generic method printArray  
        for(X element : inputArray) {      // Display array elements  
            System.out.printf("%s ", element);  
        }  
        System.out.println();  
    }  
    public static void main(String args[]) {  
        // Create arrays of Integer, Double and Character  
        Integer[] intArray = { 1, 2, 3, 4, 5 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4 };  
        Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };
```

Generic Class – Example #2

```
System.out.println("Array integerArray contains:");  
printArray(intArray); // pass an Integer array  
  
System.out.println("\nArray doubleArray contains:");  
printArray(doubleArray); // pass a Double array  
  
System.out.println("\nArray characterArray contains:");  
printArray(charArray); // pass a Character array  
}  
}
```

Output:
Array integerArray
contains:
1 2 3 4 5
Array doubleArray
contains:
1.1 2.2 3.3 4.4
Array characterArray
contains:
H E L L O

Generics Bounded Type

- There may be times when you want to **restrict the types** that can be used as type arguments in a parameterized type.
- **For example**, a method that operates on numbers might only want to **accept instances of Number or its subclasses**. This is what bounded type parameters are for.

- **Syntax**
<T extends superClassName/Type>
 - List the type parameter's name
 - Along by the extends keyword
 - And by its upper bound

Bounded Type – Example #1

```
//This example demonstrate bounded type

public class GenericDemo {

    // A Generic method with bounded type

    static <T extends Number> T genericDisplay (T one, T two) {

        return (T)(Integer)(one.intValue() + two.intValue());

    }

    public static void main(String[] args)    {           // Driver method

        // Calling generic method with Integer argument

        System.out.println(genericDisplay(11,5));

        // Calling generic method with String argument

        System.out.println(genericDisplay("Generics","Test")); //Error : Bound type: Number
```

Bounded Type – Example #1

```
// Calling generic method with double argument  
System.out.println(genericDisplay(1.0, 2.0));  
}  
}
```


Bounded Type – Example #2

```
//A Simple Java program to demonstrate bounded type  
class GenericDemo {  
    // A Generic method with bounded type  
    static <T extends Number> T add (T one, T two) {  
        String str=one.getClass().getName();  
        String str1=two.getClass().getName();  
        if((str=="java.lang.Integer") &&(str1=="java.lang.Integer")) {  
            return (T)(Integer)(one.intValue() + two.intValue());  
        } else if((str=="java.lang.Float") &&(str1=="java.lang.Float")) {  
            return (T)(Float)(one.floatValue() + two.floatValue());  
        }  
    }  
}
```

Generics in Java

Bounded Type – Example #2

```
        } else{  
            return null;  
        }  
    }  
  
    static <T> T add (T one, T two){  
        return (T)(one.toString() + two.toString());  
    }  
  
    // Driver method  
    public static void main(String[] args) {  
        System.out.println(add(3,4)); //Add 2 integers  
        System.out.println(add(7.5f,2.5f)); // Add 2 Floats  
        System.out.println(add("Generic","Method")); // Add or Concatenate 2 Strings  
    }  
}
```

Output:

7

10.0

GenericMethod

Bounded Type – Example #3

// This class only accepts type parameters as any class which extends class A or class A itself.

// Passing any other type will cause compiler time error

```
class BoundClass<T extends Class1>{  
    T objRef;  
  
    public BoundClass(T obj){  
        this.objRef = obj;  
    }  
    public void doCall(){  
        this.objRef.displayClass();  
    }  
}
```

Bounded Type – Example #3

```
class Class1 {  
    public void displayClass()    {  
        System.out.println("Inside Class1");  
    }  
}  
  
Class Class2 extends Class1 {  
    public void displayClass()    {  
        System.out.println("Inside Class2");  
    }  
}  
  
Class Class3 extends Class1 {  
    public void displayClass()    {  
        System.out.println("Inside Class3");  
    }  
}
```

Bounded Type – Example #3

```
public class BoundedClassDemo {  
    public static void main(String a[])    {  
        // Creating object of sub class Class3 and passing it to Bound as a type parameter.  
        BoundClass<Class3> c = new BoundClass<Class3>(new Class3());  
        c.doCall();  
        // Creating object of sub class Class2 and passing it to Bound as a type parameter.  
        BoundClass<Class2> b = new BoundClass<>(new Class2());  
        b.doCall();  
        // similarly passing super class Class1  
        BoundClass<Class1> a = new BoundClass<Class1>(new Class1());  
        a.doCall();  
    }  
}
```

Output:

Inside Class3

Inside Class2

Inside Class1

Generics in Java

Generics Wildcards

- The question mark (?), called the **wildcard**, represents **'unknown' type**.
- The "?" is there to **act as a placeholder** where you are allowed to pass the **different type of objects**.
- Normally, T and ? is used in generics as a placeholder. i.e <?> or <T>.
- **<?>**: It could be used while developer wants to **allow any type of object** for particular instance.
- The wildcard can be used in a **variety of situations**: as the type of a parameter, field, or local variable, sometimes as a return type
- The wildcard is never used as a type argument for a generic method invocation, a **generic class instance creation**, or a **super type**.
- **Example:**

```
public static double sum(List<? extends Number> list) {... }
```

Upper Bounded Wildcard

```
//This example demonstrate Upper bounded wildcard
import java.util.*;
public class Wildcards {

    public static void main(String[] args) {
        List<Integer> ints = new ArrayList<>();
        ints.add(3);
        ints.add(5);
        ints.add(10);
        double sum = sum(ints);
        System.out.println("Sum of ints="+sum);
    }
    public static double sum(List<? extends Number> list) {
        double sum = 0;
        for(Number n : list){
            sum += n.doubleValue();
        }
        return sum;
    }
}
```

Output:**Sum of ints = 18.0**

Generics in Java

Unbounded Wildcard

- The unbounded wildcard type is specified using the **wildcard character (?)**, for example, List<?>.

This is called a list of **unknown type**.

- There are **two scenarios** where an unbounded wildcard is a useful approach:
 - While writing a method that can be implemented using functionality provided in the Object class.
 - When the code is using methods in the generic class that don't depend on the type parameter.

Generics in Java

Unbounded Wildcard

- Sometimes, Generic method should be working with all types
 - In this case, an unbounded wildcard can be used
 - It is same as using `<? extends Object>`

```
//This example demonstrate Unbounded wildcard
public class WildCard02 {
    public static void main(String[] args) {
        List<Integer> ints = new ArrayList<>();
        ints.add(3);
        ints.add(5);
        ints.add(10);
        unBoundedWildcard(ints);
    }
    public static void unBoundedWildcard(List<?> list) {
        for(Object obj : list){
            System.out.print(obj + "::");
        }
    }
}
```

Output:

3::5::10::

Generics in Java

Lower Bounded Wildcard

- A lower bounded wildcard restricts the unknown type to be a **specific type or a super type of that type**.
- A lower bounded wildcard is expressed using the wildcard character ('?'), following by the **super keyword**, followed by its lower bound: `<? super A>`.
- We use generics wildcard (?) with super keyword and lower bound class to achieve this.
- We can pass lower bound or any super type of lower bound as an argument.

Lower Bounded Wildcard - Example

```
//This example demonstrate Lower bounded wildcard
import java.util.*;
public class WildCard03 {
    public static void main(String[] args) {
        List<Integer> ints = new ArrayList<>();
        ints.add(3);
        ints.add(5);
        ints.add(10);
        addNumbers(ints);
    }
    public static void addNumbers(List<? super Integer> list) {
        for (int i = 1; i <= 10; i++) {
            list.add(i);
        }
        System.out.println(list);
    }
}
```

Output:

[3, 5, 10, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Quiz



1. Why are generics used?

a) Generics make code more fast

b) Generics add stability to your code by making more of your bugs detectable at run time

c) Generics add stability to your code by making more of your bugs detectable at compile time

d) Generics make code more optimised and readable

c) Generics add stability to your code by making more of your bugs detectable at compile time

Quiz



2. Which of these type parameters is used for a generic class to return and accept any type of object?

a) V

b) T

c) K

d) G

b) T

Quiz



3. Which of the following reference types cannot be generic?

a) Interface

b) Inner Class

c) Anonymous Class

d) All of the above

c) Anonymous Class

Quiz



4. What are generic methods?

a) Generic methods are the methods that extend generic class's methods

b) Generic methods are methods that introduce their own type parameters

c) Generic methods are methods that take void parameters

d) Generic methods are the methods defined in a generic class

b) Generic methods are methods that introduce their own type parameters

Quiz



5. Which of these types cannot to be used to initiate a generic type?

a) Integer Class

b) Primitive Types

c) Float Class

d) Collections

b) Primitive Types

Quiz



6. Which of these data type cannot be type parameterized?

a) Set

b) Array

c) Map

d) List

b) Array

Quiz



7. Which of the following cannot be Type parameterized?

a) Overloaded Methods

b) Generic Methods

c) Class Methods

d) Overriding Methods

a) Overloaded Methods

Quiz



8. What is use of wildcards?

a) It is used for type argument of generic method

b) It is used to make code more readable

c) It is used in cases when type being operated upon is not known

d) It is used to access members of super class

c) It is used in cases when type being operated upon is not known

Quiz



9. Which of these keywords is used to upper bound a wildcard?

a) super

b) class

c) lower

d) extends

d) extends

Quiz



9. Which of these keywords is used for lower bounding a wildcard?

a) super

b) class

c) lower

d) extends

a) super

THANK YOU